

INTERNAL WORKING DOCUMENT — build plan with open decisions. Not published on the site. Companion to the public architecture doc (security-architecture.pdf , Part II).

Draft · July 2026 · Fleets: `was.moqcdn.net` (SJC) + a second region (AMS) · Control: `tinymoq.com/cdnadmin`

The mutual-edge case: two customers, two fleets, each provisioning edges for the other's origins off a shared node-id lookup. A Wallflower broadcaster's origin lives in the SJC fleet; a Vivoh viewer in Europe watches through an edge that Vivoh's AMS fleet spins up on demand — discovered purely by node-id, media pulled SJC→AMS over iroh, and symmetric (SJC can edge for AMS origins too). This document is the phased plan and the decisions to confirm; the conceptual model lives in the architecture doc Part II (§9–17).

1. Goal & demo success criteria

A broadcaster on Wallflower.tv goes live (origin in SJC). A viewer on Vivoh.Earth in Europe watches that stream through an edge Vivoh's AMS fleet provisions on demand — discovered by node-id lookup, media pulled SJC→AMS over iroh, working symmetrically.

The demo passes when:

- Video plays end to end (SJC broadcaster → AMS edge → European viewer).
- The SJC *origin* relay logs a *foreign* edge cluster-connecting and subscribing (`connected version=moq-lite-04`), dialled by resolved `host:port` .
- The AMS box — another customer's infrastructure — only ever holds **ciphertext** (E2E verified by inspection at the edge).
- **No step used a hard-coded `host:port` for the origin** — the origin was reached purely by resolved `EndpointId`.

2. Infra mapping — what is real today

In the diagram	Real box	State
SJC MoQcdn Fleet	<code>was.moqcdn.net</code>	Native iroh <code>moq-relay</code> built; relay↔relay iroh federation proven
AMS MoQcdn Fleet	new second-region box (or simulated on <code>was</code> first — Decision 4)	Reuse the same native relay binary + autoscaler
Control / routing	<code>tinymoq.com/cdnadmin</code>	Fleets register up; customer registry exists; "pass token + origin" handoff is planned, unbuilt
Shared directory	new — evolve luke's in-mem <code>/register + /resolve</code>	Must become network-shared, signed, ideally DHT-backed

3. The heart — the shared directory record

Everything hinges on one record, published by the origin and resolvable by anyone. The **broadcast NodeID** (the ed25519 pubkey the broadcaster mints; the shareable handle) is the directory *key*; the *value* is the origin relay's **host:port** — the Hermit relays are not iroh endpoints, so the record holds an address, not an EndpointId. iroh supplies the identity (node-id format) and the discovery (pkarr/DHT), not the dial target.

```
broadcast_NodeID -> {
  origin_host_port : <SJC origin relay host:port, e.g. "ams.moqcdn.net:8004">, // what the edge dial
  broadcast_name   : <moq name on that relay>,                               // what the edge subscribes to
  home_fleet      : <SJC cdnadmin URL>,                                     // control-plane owner (gated pulls
  access          : "public" | "gated",
  owner_sig       : sign(broadcast_privkey, "register:" + NodeID) // ownership proof (§16 primitive
}
```

Resolution is open (any fleet may read it — safe, it is only a locator). **Registration is authenticated** by `owner_sig`. That single signed record is simultaneously the anti-poisoning mitigation and the migration primitive (arch doc §16). Because the per-broadcast `host:port` churns as relays are spawned/reaped, the **control plane keeps the value current** (the autoscaler/Worker knows the port it assigned and re-publishes) — that is how a stable node-id solves the "dynamic origin port" problem.

Scope note on the signature. `owner_sig` proves ownership of the *broadcast NodeID*, not of the *origin host:port*. The binding "this NodeID really lives at that address" is asserted by the home fleet, not proven by the record alone. Fine for a public demo (the origin relay is the home fleet's trusted infra); revisit for gated / adversarial cases.

4. End-to-end flow (the demo scenario)

① Origin comes up — Wallflower / SJC

- Broadcaster mints the broadcast NodeID and signs `register:<id>`.
- Wallflower's control provisions an origin relay in the SJC fleet and captures that relay's `host:port`.
- Wallflower publishes the directory record above (`access:"public"` for v1).
- Camera publishes to the SJC origin relay over WebTransport. Media is **E2E-encrypted at the source** (MoQplay, broadcaster-held key) before it enters QUIC.

② Discover — Vivoh viewer

- Viewer has the broadcast NodeID (out of band, or from the shared content graph).
- Vivoh resolves NodeID → `{ SJC origin host:port, broadcast_name, access:public }`.

③ Provision the edge — Vivoh / AMS (the "edge for the other's origin" step)

- Vivoh's path asks its home AMS fleet for a watch endpoint for this NodeID.
- AMS `cdnadmin` sees the origin is remote (a `host:port` on another fleet) and provisions a Hermit **edge relay** (`spawn_edge_hermit`, `ASC_EDGE_HERMIT=1`) via native `moq-relay` cluster-connect to the resolved `<SJC origin host:port>`, plus a pull scope (public → open; gated → `&pull=<token signed with the origin's key>`).
- The AMS Hermit edge dials the SJC origin by address and subscribes to `broadcast_name` (logs: `dialing cluster peer` → `connected version=moq-lite-04`). **PROVEN MECHANISM**

④ Watch — browser ↔ AMS edge

- Viewer connects browser → AMS edge over WebTransport (view token; public → none).
- The AMS edge serves bytes pulled from SJC. Because the content is E2E-encrypted, the AMS box — a foreign customer's infra — sees only **ciphertext**; the viewer's browser holds the key and decodes.

Reverse direction (Vivoh origin, Wallflower edge) is the same flow with roles swapped — the peering is mutual.

5. Build phases

Phase	Work	Status
0 — Reuse what's proven	Hermit <code>moq-relay</code> cluster-connect by <code>host:port</code> + the bulk-RX fix (was); signed register + resolve (luke, in-mem). No new work — these are the parts.	Proven
1 — Shared directory + signed register	Promote luke's in-mem directory to a network-shared service both fleets read/write, using the record schema (§3), with <code>owner_sig</code> enforced on write. Also Wallflower's Milestone 1 (broadcasts sign-register their minted id).	New
2 — Origins publish their record	When a fleet spawns an <i>origin</i> relay, capture its <code>host:port</code> and publish (and re-publish on port churn) the NodeID→ <code>host:port</code> record. This is the "autoscaler isn't directory-wired yet" gap.	New
3 — Edge provisioning by lookup	Teach <code>cdnadmin</code> : on a watch request for a NodeID whose origin is remote, resolve → provision a Hermit edge that native-cluster-connects to the resolved <code><origin host:port></code> . The planned "route to downstream fleet, pass token + origin" — with origin = NodeID (resolved to an address).	New
4 — Cross-fleet public demo	Stand up (or simulate) AMS, onboard Vivoh.Earth as its tenant, broadcast on SJC, watch through AMS. Public streams only. Verify the four success criteria (§1).	New
5 — Gated content (deferred)	Cross-org pull-token brokering (<code>cdnadmin</code> ↔ <code>cdnadmin</code>), origin-minted view tokens, and the hard one — decryption-key distribution to a viewer who arrived via a foreign edge (concern C: possession is permanent, revocation = rekey). Out of scope for the first demo.	Deferred

Correction (2026-07-12) — the Hermit dial-out limitation was FIXED. An earlier note held that a Hermit relay could not receive sustained bulk media on a connection it dialled out, implying federating relays would need a separate iroh-enabled build. That is no longer true. The real root cause was a shallow `smoltcp` per-UDP-socket RX buffer (4 slots → 512; "100-packet burst 4/100 → 100/100") plus a nonblocking-recv / cross-core wakeup panic race — both patched. Cross-cluster edge pull now runs as a **Hermit guest** (`ASC_EDGE_HERMIT=1`), on the iroh-less serving build, dialling by `host:port`. So Phase 2 is genuinely just "capture + publish the `host:port`" — no build switch, no iroh in the relay.

6. Proven vs. new — at a glance

Proven	New (this effort)
relay↔relay Hermit cluster-connect by <code>host:port</code> — real media (225,747 bytes, 2 fleets)	Shared / DHT directory (vs in-mem)
signed register + resolve (luke); node id → <code>host:port</code> on pkarr across two owners	Autoscaler capturing + publishing the origin <code>host:port</code>
Hermit bulk-RX fix (dial-out sustained pull); <code>ASC_EDGE_HERMIT</code> edge	<code>cdnadmin</code> edge-provision-by-lookup with origin = NodeID
per-tenant fleet onboarding; managed per-stream keys	Second region box (or simulate on <code>was</code>) + the cross-fleet public demo
—	Gated cross-org tokens + key distribution — Phase 5

7. Decisions to confirm

Recommended defaults marked **REC**. All four defaults together yield the fastest real cross-customer, node-id-routed demo with every hard problem deferred behind a working skeleton. Pending Erik's confirmation; **Decision 4 is the one worth deciding first.**

#	Decision	Default	Alternative
1	Directory backend v1	REC Shared HTTP now + pkarr/DHT publish in parallel (record schema is backend-agnostic, so the swap is invisible; matches cdnadmin 's centralized shape)	Go straight to DHT
2	Edge trigger	REC Viewer-reactive (first viewer provisions the edge)	Pre-provisioned peering for hot origins
3	Scope of first demo	REC Public streams only (dissolves the B view-token and C key-distribution problems)	Include one gated stream
4	AMS box	REC Simulate the second fleet on was (a second relay) to prove the software path first	Spin a fresh region box for real geo separation

Why Decision 4 comes first. Simulating AMS on was proves everything genuinely new — second relay, cross-cluster iroh pull, directory lookup, edge provisioning — before anyone pays for a second region. Real geo separation then becomes a deployment detail, not a prerequisite. It is cheap and de-risks the software before a second box exists.

8. Auth & key modes

A tenant in `customers.json` is `{ id, key:"<provisioning bearer>", quota, name, verify_jwk? }.` On per-stream-key fleets (`ASC_PER_STREAM_KEY=1`) there are two keying modes; token claims are `{ put: [paths], get:[paths], exp }` in both.

Managed — no `verify_jwk`

The box mints a fresh ephemeral HS256 `K_stream` per broadcast per relay, maps it into that relay, and returns it from `/assign`. The caller signs its own tokens with it; reaping the relay deletes the key — cryptographic revocation for free. **Caveat:** `K_stream` differs per box, so a cross-cluster edge cannot mint a token the *remote* origin will accept — the edge request MUST pass `&pull=<token signed with the ORIGIN's K_stream>`.

BYOK — `verify_jwk` present (oct / EdDSA / ES256)

The customer's key verifies on all their relays; the box holds only the verify key and mints nothing. **One key works on origin + edge + viewer**, so the caller mints every token — no per-box key juggling.

RECOMMENDED FOR CROSS-CLUSTER (it removes the `&pull=` juggling above).

9. Operational gotchas

- **Edge reap window.** A cross-cluster edge reaps after `ASC_EDGE_IDLE_S` (=45s) with no viewer — the viewer must connect within that window (or pre-mint the token and subscribe immediately).
- **`/assign` is sticky per broadcast.** To change keying (managed↔BYOK) or config, use a NEW broadcast name.
- **Hairpin NAT.** A publisher/subscriber on a relay's own box cannot reach it via the external `IP:800X` (DNAT doesn't hairpin). Run pub/sub from a DIFFERENT box — cross-box is real external traffic and works.
- **`--cluster token claim`.** The deployed `moq-token-cli` (0.5.29) doesn't emit it (a local patch, absent upstream). A plain subscribe token works as the pull token in practice; callers pass `&pull=`.
- **Headless test tooling.** `moq-cli` (publish/subscribe, media on stdio) + `moq-token-cli`, built from `moq-dev/moq` at the relay's commit (`02273a0b`) for protocol match (moq-lite-04). Publisher = `ffmpeg fragmented-mp4 → moq-cli pub`.

10. Where iroh does — and doesn't — live

The crux, stated plainly:

- **USED:** identity (the node-id) + discovery (the pkarr/DHT directory). This is the whole of iroh's role.
- **NOT USED:** iroh *transport* for relay↔relay — native cluster-connect by `host:port` already works on Hermit post bulk-RX-fix; forcing iroh in would add DNS/threading complexity for no gain.
- **FUTURE-ONLY:** iroh transport is the enabler for a NAT'd / no-public-ingress private origin ("run your own origin behind your firewall, we federate it"). If built, it would be a native iroh gateway *per fleet*, not inside every Hermit relay. Remaining iroh-in-Hermit blockers if attempted: Hermit's single-threaded `current_thread` vs. iroh's multithread model; no working DNS + outbound-TLS; a dependency forward-port (`aws-lc → ring`). **Bulk-RX is no longer a blocker.**

